

وظيفة البرمجة التفرعية

يهدف المشروع إلى بناء منصة تجارة إلكترونية متطورة تركز على حل تحديات التزامن "Concurrency".

حيث لا يقتصر المشروع على البيع والشراء التقليدي بل يتمحور حول ضمان استقرار النظام وسلامة البيانات عند تعرضه لضغط كبير من الطلبات المتزامنة في وقت واحد.

يركز المشروع على تحقيق التوازن بين كفاءة الأداء ودقة البيانات حيث تكمن أهدافنا الأساسية في منع تضارب العمليات (Race Conditions) وتنظيم استهلاك موارد الخادم (CPU & RAM) وضمان استجابة سريعة للمستخدم حتى في أصعب ظروف التحميل المتزامن.

تتمثل الوظائف الأساسية للنظام -والتي تمثل المتطلبات الوظيفية- في إدارة عملية الشراء بدقة وتشمل:

- إدارة المخزون: تحديث كميات المنتجات في قاعدة البيانات فور إتمام كل عملية.
- معالجة الطلبات: استقبال طلبات الشراء والتحقق من توفر الكمية المطلوبة قبل الخصم.
- سجل العمليات: إنشاء سجلات دقيقة لكل طلب شراء ناجح لضمان تتبع البيانات.

هيكل قاعدة البيانات (Database Schema):

جدول المنتجات (Products): وهو الجدول المحوري الذي يحتوي على بيانات المنتجات مع حقل "المخزون (Stock)" الذي نطبق عليه عمليات القفل (Locking) لضمان دقة الكميات.

	id	name	price	stock	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	Laptop AI Edition	1200.00	1	2026-05-09 12:43:46	2026-05-15 12:08:55

جدول الطلبات (Orders): يُستخدم لتوثيق العمليات الناجحة حيث يربط كل عملية شراء بالمنتج الخاص بها ويسجل الكمية التي تم بيعها.

← T →			▼ id	user_id	total_price	status	created_at	updated_at	
☐	 Edit	 Copy	 Delete	1	1	4800.00	completed	2026-05-10 16:44:43	2026-05-10 16:44:43
☐	 Edit	 Copy	 Delete	2	1	4800.00	completed	2026-05-10 16:44:43	2026-05-10 16:44:43
☐	 Edit	 Copy	 Delete	3	1	1200.00	completed	2026-05-10 17:01:50	2026-05-10 17:01:50
☐	 Edit	 Copy	 Delete	4	1	1200.00	completed	2026-05-10 17:01:50	2026-05-10 17:01:50
☐	 Edit	 Copy	 Delete	5	1	1200.00	completed	2026-05-10 17:11:18	2026-05-10 17:11:18
☐	 Edit	 Copy	 Delete	6	1	1200.00	completed	2026-05-15 11:17:28	2026-05-15 11:17:28
☐	 Edit	 Copy	 Delete	7	1	1200.00	completed	2026-05-15 11:44:26	2026-05-15 11:44:26
☐	 Edit	 Copy	 Delete	8	1	1200.00	completed	2026-05-15 11:44:46	2026-05-15 11:44:46
☐	 Edit	 Copy	 Delete	9	1	1200.00	completed	2026-05-15 11:44:54	2026-05-15 11:44:54
☐	 Edit	 Copy	 Delete	10	1	1200.00	completed	2026-05-15 11:49:27	2026-05-15 11:49:27

جدول المهام (Jobs): وهو الجدول المسؤول عن دعم المعالجة غير المتزامنة حيث تُخزن فيه المهام الثقيلة (مثل تأكيد الطلب) مؤقتاً حتى يتم معالجتها في الخلفية.

						id	queue	payload	attempts	reserved_at	available_at	created_at
	Edit	Copy	Delete	8	default	{\"uuid\":\"9498e969-36b9-4bab-ad45-41ff90eb39f2\",\"di...			1	1778846836	1778846836	1778846836

جدول تفاصيل الطلبات (Order Items): هو الجدول الذي يربط الطلب بالمنتج، حيث يسجل لكل طلب ما هي المنتجات التي تم شراؤها وبأي كمية وبأي سعر في تلك اللحظة.

← T →			▼	id	order_id	product_id	quantity	price	created_at	updated_at
☐	 Edit	 Copy	 Delete	1	1	1	4	1200.00	2026-05-10 16:44:43	2026-05-10 16:44:43
☐	 Edit	 Copy	 Delete	2	2	1	4	1200.00	2026-05-10 16:44:43	2026-05-10 16:44:43
☐	 Edit	 Copy	 Delete	3	3	1	1	1200.00	2026-05-10 17:01:50	2026-05-10 17:01:50
☐	 Edit	 Copy	 Delete	4	4	1	1	1200.00	2026-05-10 17:01:50	2026-05-10 17:01:50
☐	 Edit	 Copy	 Delete	5	5	1	1	1200.00	2026-05-10 17:11:18	2026-05-10 17:11:18
☐	 Edit	 Copy	 Delete	6	6	1	1	1200.00	2026-05-15 11:17:28	2026-05-15 11:17:28
☐	 Edit	 Copy	 Delete	7	7	1	1	1200.00	2026-05-15 11:44:26	2026-05-15 11:44:26
☐	 Edit	 Copy	 Delete	8	8	1	1	1200.00	2026-05-15 11:44:46	2026-05-15 11:44:46
☐	 Edit	 Copy	 Delete	9	9	1	1	1200.00	2026-05-15 11:44:54	2026-05-15 11:44:54
☐	 Edit	 Copy	 Delete	10	10	1	1	1200.00	2026-05-15 11:49:27	2026-05-15 11:49:27
☐	 Edit	 Copy	 Delete	11	11	1	1	1200.00	2026-05-15 12:07:08	2026-05-15 12:07:08
☐	 Edit	 Copy	 Delete	12	12	1	1	1200.00	2026-05-15 12:07:16	2026-05-15 12:07:16
☐	 Edit	 Copy	 Delete	13	13	1	1	1200.00	2026-05-15 12:08:55	2026-05-15 12:08:55

معالجة المتطلبات الغير وظيفية:

١. الطلب الأول:

سلامة البيانات والتزامن (Data Integrity & Concurrency):

في أنظمة المتجر التقليدية، إذا كان هناك قطعة واحدة فقط من منتج ما وحاول شخصان الضغط على زر "شراء" في نفس الأجزاء من الثانية قد يقرأ النظام لكليهما أن الكمية متاحة فيقوم ببيع القطعة مرتين ويصبح المخزون بالسالب.

لحل هذه المشكلة استخدمنا تقنية Pessimistic Locking عبر دالة lockForUpdate()

```
$product = Product::where('id', $productId)->lockForUpdate()->first();
```

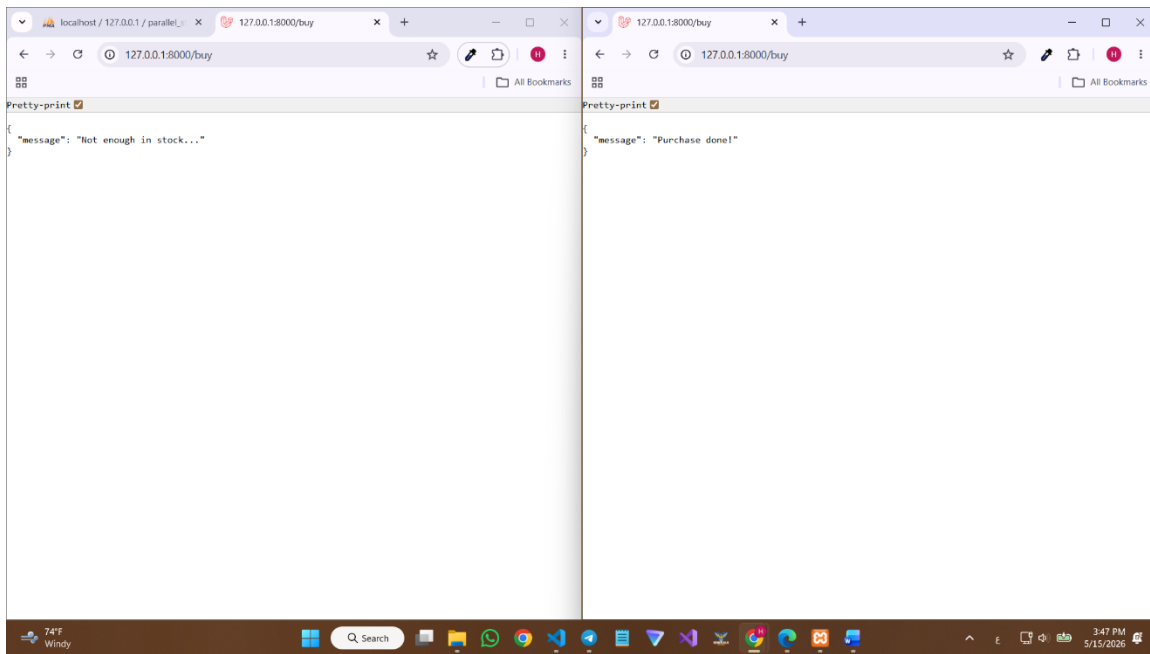
عندما يبدأ المستخدم الأول عملية الشراء يقوم السيرفر بوضع قفل (Lock) على سطر المنتج في قاعدة البيانات حيث إذا حاول المستخدم الثاني الدخول في نفس اللحظة سيجده مقفلاً فيضطر للانتظار حتى تنتهي العملية الأولى تماماً.

الاختبار:

- تجهيز بيئة الاختبار بضبط كمية المخزون لمنتج معين إلى (١) فقط لاختبار استجابة النظام عند محاولة سحب كميات تفوق المتاح تحت الضغط المتزامن.

	id	name	price	stock	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	Laptop AI Edition	1200.00	1	2026-05-09 12:43:46	2026-05-15 12:08:55

- بعد الضغط على زر الشراء في الشاشتين بشكل متوازي يظهر نجاح الطلب الأول أقصى اليمين وفشل الطلب الثاني أقصى اليسار برسالة "الكمية غير كافية" مما يثبت فعالية ال-Pessimistic Locking في منع البيع الوهمي.



- يظهر تحديث المخزون إلى القيمة (٠) بدقة مع تسجيل طلب واحد فقط في جدول العمليات، مما يؤكد عدم حدوث أي تضارب.

	id	name	price	stock	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	Laptop AI Edition	1200.00	0	2026-05-09 12:43:46	2026-05-15 12:47:19

	id	queue	payload	attempts	reserved_at	available_at	created_at
☐ Edit ☐ Copy ☐ Delete	13	default	{"uuid":"a8e895de-1031-4796-a96c-c2fb51d7406","di...	1	1778849537	1778849536	1778849536

او يمكن الاختبار عن طريق تشغيل ملف السكريبت
test_race_condition_and_rate_limiter.ps1
الذي يقوم بضرب طالين بنفس اللحظة

```
=====
Launching 2 Concurrent Requests for 1 Available Stock...
=====
Processing, waiting for both jobs to finish...
[SUCCESS] {"message":"Purchase complete..","source":"MySQL updated & Redis synced","remaining_stock":0}
[REJECTED] {"message":"Out of stock / Request Blocked"}
```

- فنجد ان الأول قد تم بنجاح وانخفضت قيمة المخزون من ١ الى ٠ والثاني فشل

٢. الطلب الثاني:

إدارة السعة وحماية النظام (Capacity & Rate Limiting):

عندما يقوم مستخدم واحد (أو بوت) بإرسال مئات الطلبات في ثانية واحدة قد يتسبب ذلك في إغراق السيرفر وقاعدة البيانات مما يؤدي لبطء شديد أو توقف الخدمة عن بقية المستخدمين .

لحل هذه المشكلة استخدمنا Rate Limiter Middleware يقوم بعدّ الطلبات القادمة من كل مستخدم بناءً على عنوان IP الخاص به ، إذا تجاوز المستخدم الحد المسموح به يتم حظره مؤقتاً ومنع طلبه من الوصول للكود الأساسي أو قاعدة البيانات. قمنا ببرمجة النظام ليسمح بـ 5 طلبات فقط في الدقيقة.

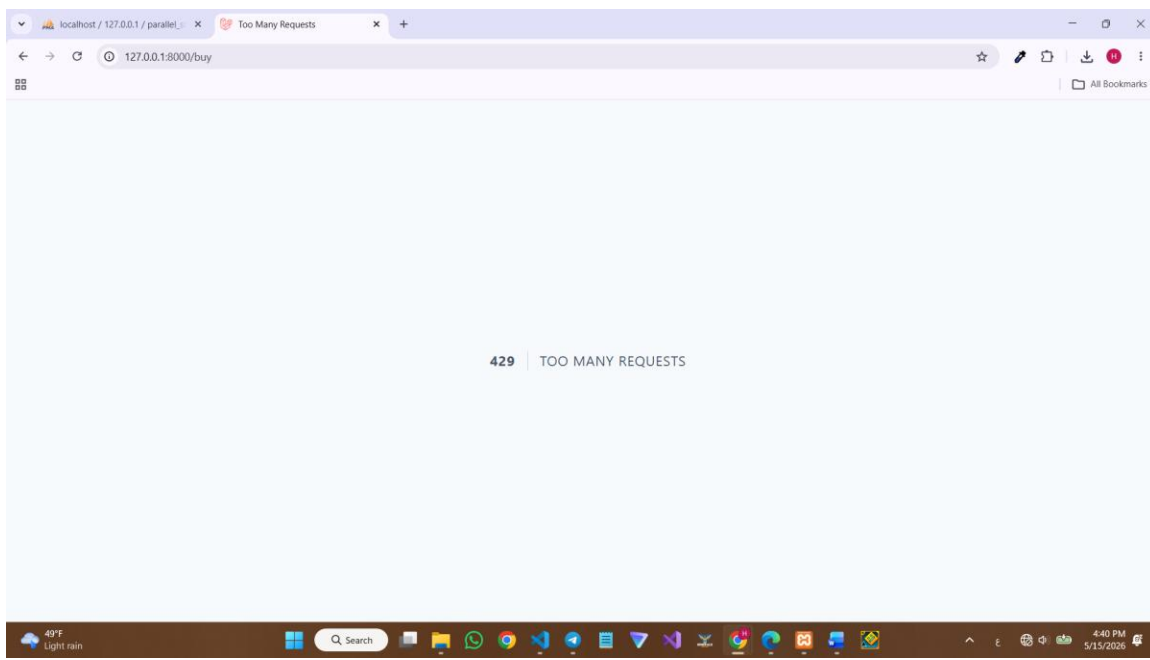
```
public function boot(): void
{
    RateLimiter::for('purchase_limit', function (Request $request) {
        return Limit::perMinute(5)->by($request->ip());
    });
}
```

الاختبار:

- نرفع عدد المنتجات في ال stock الى ٦ حتى نختبر الضغط على زر الشراء ٦ مرات

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

- نضغط على زر الشراء ٦ مرات فنجد ان النظام استجاب للضغوطات ال ٥ الأولى ولكن عند الضغطة السادسة تظهر شاشة الخطأ بأن هناك الكثير من الطلبات حدثت خلال هذه الدقيقة



- نجد أن العدد في ال stock أصبح ١ بدل ان يصبح ٠ على الرغم من الضغط ٦ مرات

	id	name	price	stock	created_at	updated_at
☐ Edit Copy Delete	1	Laptop AI Edition	1200.00	1	2026-05-09 12:43:46	2026-05-15 13:40:41

```

2026-05-15 16:43:81 /buy ..... ~ 581.33ms
2026-05-15 16:43:81 /buy ..... ~ 8.15ms
2026-05-15 16:43:81 /buy ..... ~ 4.41ms
2026-05-15 16:43:81 /buy ..... ~ 581.88ms
2026-05-15 16:43:81 /buy ..... ~ 496.29ms
2026-05-15 16:43:82 /buy ..... ~ 1.06ms
2026-05-15 16:43:82 /favicon.ico ..... ~ 1.06ms

```

او يمكن الاختبار عن طريق تشغيل ملف السكريبت test-rate-limiter.ps1 حيث نقوم برفع الحد الى ٢٤ طلب بدلا من ٦, ويقوم السكريبت بضرب ٢٦ طلب شراء

فنجذ ان الطلبات ال ٢٤ الأولى تمت بنجاح بينما فشل طلبان

```
=====
Launching 26 Concurrent Requests to Break Rate Limiter (Limit: 24)...
=====
Firing requests simultaneously, holding thread for results...
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":29}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":28}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":27}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":26}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":25}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":24}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":23}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":22}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":21}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":20}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":19}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":18}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":17}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":16}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":15}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":14}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":13}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":12}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":11}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":10}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":9}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":8}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":7}
[ALLOWED] {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":6}
[FAILED] Connection Error
[FAILED] Connection Error
```

٣. الطلب الثالث:

الأداء والمعالجة المؤجلة (Performance - Asynchronous Queues):

هناك مهام ثقيلة برمجياً مثل إرسال إيميل تأكيد أو تحديث سجلات إحصائية ضخمة أو محاكاة عملية معالجة تستغرق وقتاً.

إذا نفذنا هذه المهام مباشرة أثناء طلب الشراء سيضطر المستخدم للانتظار لثوانٍ طويلة أمام شاشة بيضاء قبل أن يعرف هل نجحت العملية أم لا مما يؤدي لتجربة مستخدم سيئة.

لحل هذه المشكلة استخدمنا نظام الطوابير "Queues" ف بدلاً من تنفيذ المهمة الثقيلة فوراً يقوم النظام بإنشاء تذكرة للمهمة ويضعها في جدول jobs في قاعدة البيانات ثم يُرسل النظام رداً فورياً للمستخدم بكلمة "تم النجاح." عن طريق تابع Dispatch على الرغم من وجود sleep لمدة ٥ ثوان داخل كلاس

PeocessOrderConfirmation


```
ProcessOrderConfirmation::dispatch($order);
```

```
public function handle(): void  
{  
    sleep(5);  
    \Log::info("Proccess done in the background");  
}
```

يقوم محرك خلفي (Queue Worker) بمعالجة هذه المهام واحدة تلو الأخرى بعيداً عن عين المستخدم.

الاختبار:

- نقوم بإيقاف المحرك عبر تعليمة `php artisan queue:clear`

```
C:\UNI Projects\parallel-store>php artisan queue:clear  
  
INFO Cleared 11 jobs from the [default] queue.
```

- نضغط على زر الشراء ف ستظهر رسالة النجاح فوراً في اقل من ثانية رغم وجود تأخير ٥ ثوانٍ في الكود.

```
{  
    "message": "Purchase done!"  
}
```

- سنجد سطرأ جديداً ظهر هناك، وهذا يعني أن المهمة منتظرة ولم يتم تنفيذها بعد.

	id	queue	payload	attempts	reserved_at	available_at	created_at
	25	default	["uuid":"ed9e6fca-b916-47db-b7a9-e8f94624b31f","di...	0	NULL	1778853552	1778853552

- نقوم بتفعيل محرك الطوابير عبر الأمر `php artisan queue:work` فنجد ان المهمة يتم تنفيذها الذي يستغرق ٥ ثوان بسبب وجود `sleep`.

```
C:\UNI Projects\parallel-store>php artisan queue:work  
2026-05-15 14:02:58 App\Jobs\ProcessOrderConfirmation ..... RUNNING  
2026-05-15 14:03:03 App\Jobs\ProcessOrderConfirmation ..... 55 DONE
```

او يمكن تنفيذ الاختبار عن طريق تشغيل ملف السكريبت `test_async_queues.ps1` الذي يقوم بضرب ١٠ طلبات شراء وحساب الوقت الكلي المستغرق للتنفيذ حيث ١٠ طلبات يجب ان تأخذ ٥٠ ثانية بسبب وجود `sleep` لمدة ٥ ثوان لكل طلب


```

=====
Sending 10 Fast Requests to test Asynchronous Queues...
=====
[SERVER RESPONSE 1]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":9}
[SERVER RESPONSE 2]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":8}
[SERVER RESPONSE 3]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":7}
[SERVER RESPONSE 4]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":6}
[SERVER RESPONSE 5]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":5}
[SERVER RESPONSE 6]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":4}
[SERVER RESPONSE 7]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":3}
[SERVER RESPONSE 8]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":2}
[SERVER RESPONSE 9]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":1}
[SERVER RESPONSE 10]: {"message":"Purchase complete...","source":"MySQL updated & Redis synced","remaining_stock":0}
=====
Total script execution time: 1.7017552 seconds
=====
Check your Terminal running 'php artisan queue:work' to see background job processing!

```

ولكن نجد ان الوقت الكلي هو اقل من ثانيتين بسبب وجود تابع الDispatch
ونلاحظ ظهور ١٠ اسطر في جدول jobs في الداتابيس تنتظر تشغيل php artisan
queue:work لكي يتم معالجتها واحد تلو الاخر مستغرة ٥٠ ثانية

٤. الطلب الرابع:

المعالجة المجمعّة المتوازية (Parallel Batch Processing)
ند التعامل مع كميات ضخمة من البيانات (مثل معالجة آلاف طلبات المبيعات اليومية لتوليد
تقارير إحصائية)، فإن معالجتها بشكل فردي تستهلك وقتاً طويلاً وتستنزف موارد قاعدة
البيانات. تم استخدام تقنية **Batching** في Laravel لتقسيم العمليات الكبيرة إلى دفعات
(Chunks) ومعالجتها بالتوازي.

التنفيذ البرمجي :

قمنا بإنشاء Job يسمى **ProcessDailySalesBatch** يقوم بتقسيم البيانات إلى مجموعات
صغيرة، واستخدام دالة **Bus::batch** لتنفيذ هذه المجموعات بشكل متوازٍ، مما يقلل الوقت
الإجمالي للمعالجة بنسبة كبيرة.

```

LoadBalancerMiddleware.php 16
Jobs
ProcessDailySalesBatch.php 17
ProcessOrderConfirmation.php 18
Models 19
Providers 20
bootstrap 21
cache 22
app.php 23
providers.php 24
config 25
database 26
public 27
resources 28
routes 29
storage 30
app 31
framework 32
logs 33
.gitignore 34
laravel.log 35
tests 36
vendor 37
editorconfig 38
env 39
16 {
17 //
18 }
19 /**
20  * Execute the job.
21  */
22 public function handle(): void
23 {
24 //
25 //
26 //App\Models\Order::whereDate('created_at', now()->today())
27 //->chunk(100, function ($orders) {
28 //    $batchJobs = [];
29 //
30 //    foreach ($orders as $order) {
31 //        $batchJobs[] = new App\Jobs\ProcessOrderConfirmation($order);
32 //    }
33 //
34 //    if (count($batchJobs) > 0) {
35 //        \Illuminate\Support\Facades\Bus::batch($batchJobs)->dispatch();
36 //        \Illuminate\Support\Facades\Log::info("تم إطلاق دفعة معالجة متوازية لـ " . count($batchJobs) . " طلب.");
37 //    }
38 //});
39 //
40 //if (App\Models\Order::whereDate('created_at', now()->today())->count() == 0) {
41 //    \Illuminate\Support\Facades\Log::warning("لم يتم العثور على طلبات جديدة لمعالجتها اليوم.");
42 //}
43 //
44 //}

```

الاختبار والنتائج: عند تشغيل الرابط المخصص للاختبار، يقوم النظام بإطلاق مجموعة من المهام في الخلفية. يمكن ملاحظة النتائج في جدول `job_batches` في قاعدة البيانات الذي يوضح حالة كل دفعة ونسبة الإنجاز، بالإضافة إلى ملف السجلات (Logs) الذي يظهر بدء وانتهاء كل مجموعة.

finished_at	created_at	cancelled_at	options	failed_job_ids	failed_jobs	pending_jobs	total_jobs	name	id
1778836336	1778836132	NULL	{:a:0}	0	0	100	100	a1c8c193-6987-4290-9042-910cca58fb11	حذف نسخ تعديل
1778836540	1778836132	NULL	{:a:0}	0	0	100	100	a1c8c194-6b62-4ac3-9fa0-015bf11a15fd	حذف نسخ تعديل
1778873353	1778873149	NULL	{:a:0}	0	0	100	100	a1c99e37-c4cd-431d-9888-1e8e7e1b4c39	حذف نسخ تعديل
1778873557	1778873149	NULL	{:a:0}	0	0	100	100	a1c99e37-ce5e-4895-8ab3-08a11c48811f	حذف نسخ تعديل

٥. الطلب الخامس:

محاكاة توزيع الأحمال (Load Balancing Simulation)
المفهوم البرمجي: لضمان استقرار النظام عند هجوم عدد كبير من المستخدمين في وقت واحد، يجب توزيع الطلبات على أكثر من خادم (Server).

في هذا المشروع، قمنا بمحاكاة وجود خادمين (Server A و Server B) وتوزيع الطلبات بينهما لضمان عدم انهيار خادم واحد تحت الضغط.
التنفيذ البرمجي:

استخدمنا `LoadBalancerMiddleware` مخصص يسمى `Middleware`.

```

->withMiddleware(function (Middleware $middleware) {

    $middleware->validateCsrfTokens(except: [
        '/buy',
    ]);

    $middleware->alias([
        'load.balancer' => \App\Http\Middleware\LoadBalancerMiddleware::class
    ]);

    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
}

```

يعمل هذا الوسيط كشرطي مرور؛ يستقبل الطلب القادم من المتصفح، ويقوم باختيار أحد الخادمين الوهميين بناءً على خوارزمية توزيع عشوائي، ثم يوجه الطلب إليه.

الاختبار والنتائج:

عند القيام بعدة عمليات تحديث (Refresh) للمتصفح، نلاحظ في ملف storage/logs/laravel.log أن النظام يطبع رسائل مختلفة توضح الخادم الذي استلم الطلب:

- Load Balancer: Request directed to Server_A

- Load Balancer: Request directed to Server_B

```
[2026-05-15 19:25:47] local.INFO: Load Balancer: تم توجيه الطلب إلى الخادم Server_B
[2026-05-15 19:25:49] local.INFO: طلب 100 تم إطلاق دفعة معالجة متوازية لـ
[2026-05-15 19:25:49] local.INFO: طلب 100 تم إطلاق دفعة معالجة متوازية لـ
```

تم أيضاً إضافة رأس (Header) في استجابة المتصفح يسمى X-Server-Id ليوضح للمطور أي سيرفر قام بالخدمة.

٦. الطلب السادس:

تحسين أداء استرجاع البيانات عبر التخزين المؤقت (Product Listing Caching)

تعتبر صفحة عرض المنتجات من أكثر الصفحات استدعاءً من قبل المستخدمين.

إذا كان النظام يقوم بطلب القائمة من قاعدة البيانات مع كل إعادة تحميل للصفحة سيؤدي ذلك إلى إبطاء زمن الاستجابة.

تم تطبيق نمط Cache-Aside لحل هذه المشكلة عبر خطوتين:

١. عند الطلب الأول (Cache Miss) : يبحث النظام في Redis ، فلا يجد البيانات، فيضطر للذهاب إلى MySQL وجلب المنتجات، ثم يقوم بحفظها وتخزينها في كاش Redis قبل إرسالها للمستخدم.

٢. عند الطلب الثاني (Cache Hit) : يجد النظام القائمة جاهزة ومخزنة في الـ RAM (Redis) ، فيرجعها فوراً للمستخدم خلال ميكروثوانٍ دون تكبد عناء استعلامات قاعدة البيانات.

```

class ProductController extends Controller
{
    public function index(\Illuminate\Http\Request $request)
    {
        $cacheKey = 'all_products_list';

        if ($request->has('clear_cache')) {
            Cache::forget($cacheKey);
        }

        $isCached = Cache::has($cacheKey);
        $products = Cache::remember($cacheKey, 86400, function () {
            return Product::all();
        });

        $source = $isCached ? 'Fetched from Redis RAM (Cache Hit - Super Fast!)'
        : 'Fetched from MySQL Database (Cache Miss - First Time)';
        return response()->json([
            'status' => 'success',
            'source' => $source,
            'count' => $products->count(),
            'data' => $products
        ]);
    }
}

```

الاختبار:

- نقوم أولاً بتنظيف ذاكرة الكاش عبر تعليمة `php artisan cache:clear`

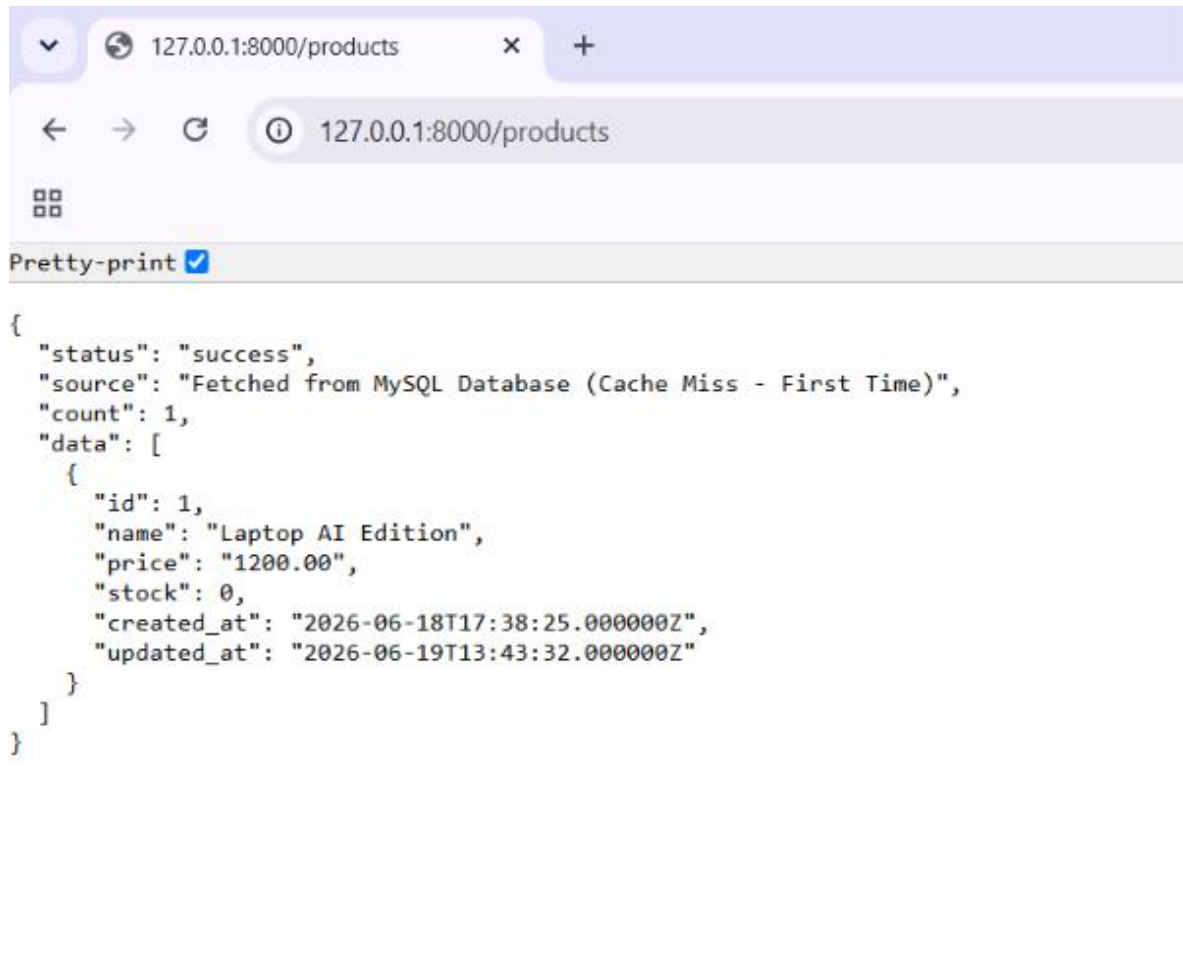
```

C:\UNI Projects\Parallel_programming_project-main>php artisan cache:clear

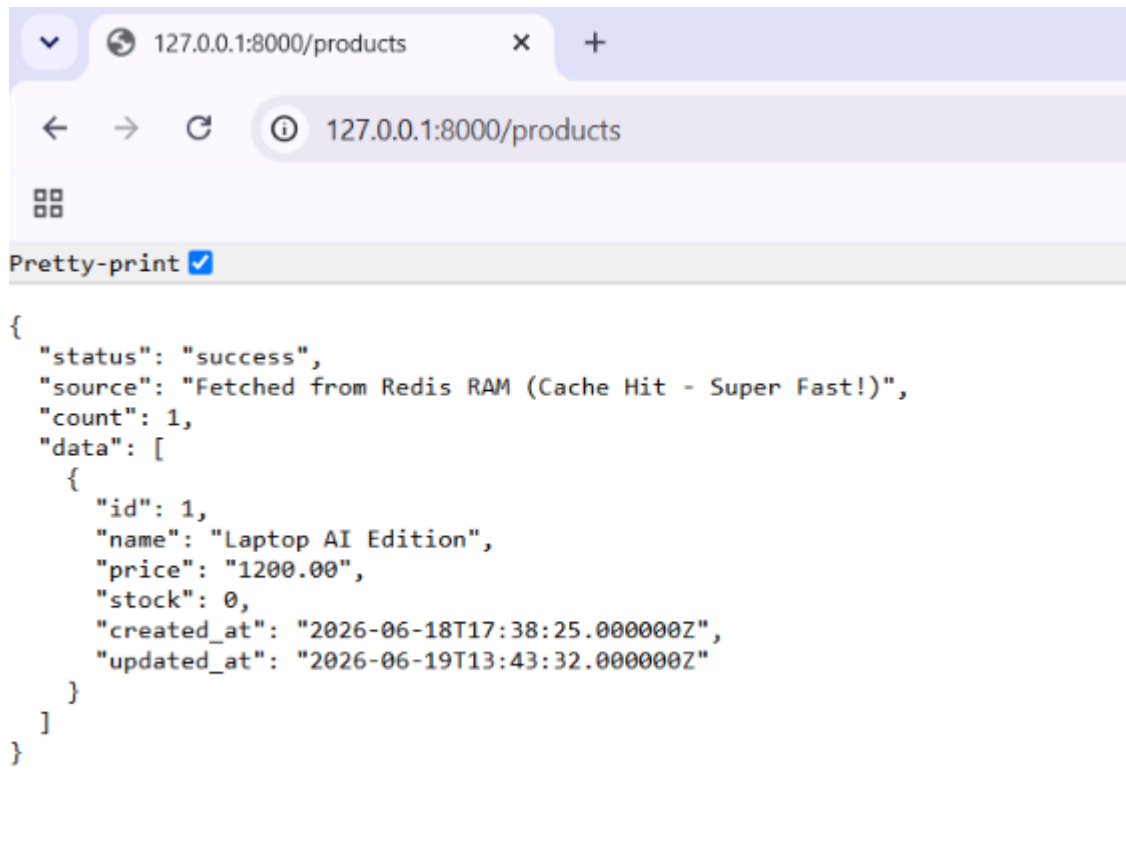
```

INFO Application cache cleared successfully.

- ثم نقوم بطلب عرض المنتجات من جدول Products عبر فتح هذا الرابط بالمتصفح <http://127.0.0.1:8000/products> فتظهر لدينا هذه الصفحة بعد بضع ثوان



- نقوم بعمل تحديث (Refresh) للصفحة فتظهر لنا هذه الصفحة بعد أجزاء من الثانية



```
{
  "status": "success",
  "source": "Fetched from Redis RAM (Cache Hit - Super Fast!)",
  "count": 1,
  "data": [
    {
      "id": 1,
      "name": "Laptop AI Edition",
      "price": "1200.00",
      "stock": 0,
      "created_at": "2026-06-18T17:38:25.000000Z",
      "updated_at": "2026-06-19T13:43:32.000000Z"
    }
  ]
}
```

أو يمكن تطبيق الاختبار عن طريق تشغيل ملف السكريبت test_products_caching

حيث يقوم هذا الملف بعمل Cache clear ثم ضرب طلبي عرض المنتجات فيكون الأول من الداتابيس والثاني من الرام مع عرض الوقت المستغرق لكل طلب

```
=====
Testing Product Listing Caching (Cache-Aside Pattern)
=====

[1] Launching FIRST request (Forcing Cache MISS)...
--- DISPLAYING PRODUCTS RECEIVED ---
â€¢ ID: 1 | Name: Laptop AI Edition | Price: 1200.00$
-----
--> Source: Fetched from MySQL Database (Cache Miss - First Time)
--> Execution Time: 189.9518 ms

[2] Launching SECOND request (Expecting Cache HIT)...
--- DISPLAYING PRODUCTS RECEIVED ---
â€¢ ID: 1 | Name: Laptop AI Edition | Price: 1200.00$
-----
--> Source: Fetched from Redis RAM (Cache Hit - Super Fast!)
--> Execution Time: 117.329 ms

=====
Notice the massive drop in Execution Time!
=====
```

ونلاحظ الفارق بالزمن بين الطلبين.

٧. الطلب السابع:

حماية المعاملات الحساسة عبر الأقفال الموزعة (Redis Distributed Lock)

عندما يضغط مئات المستخدمين على زر شراء في نفس الميكروثانية والمنتج المتبقي منه قطعة واحدة فقط تدخل الطلبات كلها في حالة تسابق. (Race Condition)

إذا تم استخدام الفحص التقليدي بدون أقفال، ستقرأ كل الخيوط البرمجية (Threads) المتزامنة نفس القيمة القديمة للمخزون (stock = 1) قبل أن يلحق أي طلب تعديلها، والنتيجة هي الموافقة على جميع عمليات الشراء، مما يؤدي إلى سحب مخزون بالسالب وإنشاء طلبات وهمية ليس لها وجود في المخزن الفعلي.

لحل هذه المشكلة تم بناء آلية القفل الموزع باستخدام دالة Cache::lock() المعتمدة على خادم Redis المشترك.

تتمحور الفكرة حول إنشاء قفل فريد لكل منتج حيث الطلب الذي يصل أولاً بميكروثانية واحدة يستحوذ على هذا القفل، ويقوم بإغلاق البوابة خلفه تماماً، مما يجبر أي طلب متزامن آخر يأتي في نفس اللحظة على الارتداد فوراً برسالة رفض، دون السماح له حتى بالاقتراب من قاعدة البيانات.

```
public function purchaseWithDistributedLock(Request $request)
{
    $productId = $request->input('product_id', 1);

    $lock = cache::lock('lock:product:' . $productId, 5);
```

تكون مدة القفل ٥ ثوان فقط لتجنب حادثة ال Deadlock في حال تعطل السيرفر

```
if ($lock->get()) {
    try {
        $purchaseResult = DB::transaction(function () use ($productId, &$remainingStock) {

            $product = Product::where('id', $productId)->lockForUpdate()->first();

            if (!$product || $product->stock <= 0) {
                return false;
            }

            $product->decrement('stock', 1);

            $product->refresh();
            $remainingStock = $product->stock;

            $totalPrice = $product->price * 1;
            Order::create([
                'product_id' => $product->id,
                'user_id' => 1,
                'total_price' => $totalPrice,
                'status' => 'completed'
            ]);

            return true;
        });
    }
}
```

في حال كان المنتج مستحوذاً على القفل تتم عملية الشراء.

```
} finally {
    $lock->release();
}
```

ثم يتم تحرير القفل عند انتهاء العملية.

الاختبار:

يوجد ملفي اختبار `test_distributed_lock` هو ملف اختبار عملية الشراء بوجود الاقفال
و ملف `test-without-lock` هو ملف اختبار عملية الشراء بدون اقفال.

عند تشغيل ملف `test_distributed_lock` يقوم هذا الملف بضرب ٢٠ عملية شراء بنفس اللحظة

```
=====
Launching 20 Concurrent Requests with Redis Distributed Lock
=====
Processing and streaming results immediately...

[SUCCESS] Request #1 -> Purchase complete safely with Redis Distributed Lock! | Stock left: 3
[SUCCESS] Request #2 -> Purchase complete safely with Redis Distributed Lock! | Stock left: 2
[SUCCESS] Request #3 -> Purchase complete safely with Redis Distributed Lock! | Stock left: 1
[SUCCESS] Request #4 -> Purchase complete safely with Redis Distributed Lock! | Stock left: 0
[REJECTED] Request #5 -> Out of stock / Request Blocked
[REJECTED] Request #6 -> Out of stock / Request Blocked
[REJECTED] Request #8 -> Out of stock / Request Blocked
[REJECTED] Request #7 -> Out of stock / Request Blocked
[REJECTED] Request #11 -> Out of stock / Request Blocked
[REJECTED] Request #16 -> Out of stock / Request Blocked
[REJECTED] Request #18 -> Out of stock / Request Blocked
[REJECTED] Request #9 -> Out of stock / Request Blocked
[REJECTED] Request #12 -> Out of stock / Request Blocked
[REJECTED] Request #13 -> Out of stock / Request Blocked
[REJECTED] Request #14 -> Out of stock / Request Blocked
[REJECTED] Request #10 -> Out of stock / Request Blocked
[REJECTED] Request #15 -> Out of stock / Request Blocked
[REJECTED] Request #17 -> Out of stock / Request Blocked
[REJECTED] Request #19 -> Out of stock / Request Blocked
[REJECTED] Request #20 -> Out of stock / Request Blocked

=====
All 20 requests processed in real-time execution!
=====
```

نلاحظ نجاح الطلبات ال ٤ الأولى وفشل الباقي بسبب نفاد الكمية.

وعند تشغيل ملف `test-without-lock` الذي يقوم بضرب ٢٠ عملية شراء ولكن بدون اقفال
عبر الدالة :

```

public function purchaseWithoutLock(Request $request)
{
    $productId = $request->input('product_id', 1);
    $product = Product::find($productId);

    if (!$product || $product->stock <= 0) {
        return response()->json([
            'status' => 'rejected',
            'message' => 'Out of stock / Request Blocked (Traditional Check)'
        ],
        422);
    }

    $product->decrement('stock', 1);

    Order::create([
        'product_id' => $product->id,
        'user_id' => 1,
        'total_price' => $product->price,
        'status' => 'completed'
    ]);

    return response()->json([
        'status' => 'success',
        'message' => 'Purchase processed WITHOUT LOCK!',
        'remaining_stock' => $product->stock - 1
    ]);
}

```

التي تقوم بعملية شراء ولكن بدون وضع اقفال.

فلاحظ عند تشغيل الملف هبوط الكمية الى ١ - عند الطلب الثالث

```
=====
🚀 Launching 20 Concurrent Requests WITHOUT Distributed Locks
=====
```

```
Firing requests simultaneously...
```

```
Request #1 -> STATUS: 200 | Body: {"status":"success","message":"Purchase processed WITHOUT LOCK!","remaining_stock":1}
Request #2 -> STATUS: 200 | Body: {"status":"success","message":"Purchase processed WITHOUT LOCK!","remaining_stock":0}
Request #3 -> STATUS: 200 | Body: {"status":"success","message":"Purchase processed WITHOUT LOCK!","remaining_stock":-1}
Request #4 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #5 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #6 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #7 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #8 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #9 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #10 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #11 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #12 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #13 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #14 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #15 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #16 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #17 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #18 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #19 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
Request #20 -> ERROR STATUS: 422 | Message: {"status":"rejected","message":"Out of stock √ Request Blocked (Traditional Check)"}
=====
```

8_ الطلب الثامن: سلامة المعاملات (ACID / Transaction Integrity)

عند تنفيذ عملية شراء في نظام التجارة الإلكترونية، فإن العملية لا تعتبر مجرد تعديل واحد على قاعدة البيانات، بل هي مجموعة عمليات مترابطة يجب أن تنجح جميعها أو تفشل جميعها
فعملية الشراء تتضمن عدة خطوات حساسة:

- التحقق من توفر كمية المنتج في المخزون .
- إنقاص الكمية المطلوبة من جدول المنتجات .
- إنشاء سجل جديد للطلب في جدول Orders.
- إنشاء تفاصيل المنتجات المشتراة في جدول Order Items.

في حال حدوث أي خطأ أثناء تنفيذ إحدى هذه العمليات، فإن استمرار باقي العمليات قد يؤدي إلى حالة غير صحيحة في البيانات، مثل خصم كمية من المخزون بدون إنشاء طلب أو تسجيل طلب بدون تحديث المخزون.

لحل هذه المشكلة تم استخدام مفهوم Database Transaction من خلال الدالة:

DB::transaction ()

حيث يتم تجميع جميع العمليات السابقة ضمن معاملة واحدة (Transaction) بحيث يضمن النظام خاصية Atomicity من خصائص ACID أي أن العملية تكون:

Commit: عند نجاح جميع الخطوات يتم تثبيت التغييرات في قاعدة البيانات .

Rollback : عند حدوث أي خطأ يتم إلغاء جميع التعديلات وإرجاع قاعدة البيانات للحالة السابقة.

كما تم دمج الـ Transaction مع آلية القفل: **lockForUpdate()**

لضمان عدم تعديل نفس المنتج من أكثر من طلب في نفس اللحظة، وبالتالي منع حدوث تضارب في قيم المخزون.

التنفيذ البرمجي:

تم تطبيق الـ Transaction داخل خدمة معالجة الطلبات:

OrderService.php

حيث تبدأ عملية الشراء داخل كتلة Transaction واحدة، ثم يتم تنفيذ الخطوات بالترتيب:

١. الحصول على المنتج مع قفل السجل داخل قاعدة البيانات .

٢. التأكد من توفر الكمية المطلوبة .

٣. تحديث قيمة Stock.

٤. إنشاء سجل الطلب .

٥. إنشاء تفاصيل الطلب .

في حال نجاح جميع العمليات يتم تنفيذ:

Commit

أما في حال ظهور أي استثناء أثناء التنفيذ يتم:

Rollback

وبذلك تبقى قاعدة البيانات في حالة صحيحة بدون بيانات ناقصة أو متعارضة.

الاختبار والنتائج:

تم اختبار سلامة المعاملة عن طريق محاكاة حدوث خطأ أثناء عملية الشراء.

خطوات الاختبار:

- تجهيز منتج يحتوي على كمية محددة في المخزون .
- إرسال طلب شراء يقوم بتعديل المخزون وإنشاء طلب .
- إحداث خطأ أثناء تنفيذ إحدى خطوات العملية .

النتيجة:

- لم يتم تسجيل الطلب في جدول Orders.
 - لم يتم تغيير قيمة المخزون .
 - تم إرجاع قاعدة البيانات للحالة السابقة بشكل كامل .
- وهذا يثبت أن النظام يطبق مفهوم Transaction Integrity ويحافظ على خصائص ACID حتى أثناء حدوث الأخطاء أو الضغط المتزامن.

9_ الطلب التاسع: اختبار الاستقرار تحت الضغط (Stress Testing)

عند زيادة عدد المستخدمين الذين يتعاملون مع النظام في نفس الوقت، قد تظهر مشاكل في الأداء مثل زيادة زمن الاستجابة أو حدوث ضغط كبير على الخادم وقاعدة البيانات.

لذلك تم إجراء اختبار ضغط لمحاكاة عدد كبير من المستخدمين المتزامنين بهدف التأكد من قدرة النظام على الاستمرار بالعمل دون انهيار أو فقدان بيانات.

التنفيذ البرمجي:

تم استخدام أداة:

K6 Load Testing

لإنشاء مستخدمين افتراضيين يقومون بإرسال طلبات متزامنة للنظام.

تم إعداد الاختبار بحيث يحاكي:

- عدد المستخدمين :

100 Virtual Users

- مدة الاختبار :

1 Minute

- إرسال عمليات شراء وطلبات مختلفة على النظام .

خلال الاختبار تم مراقبة:

- زمن الاستجابة Response Time.

- عدد الطلبات الناجحة .

- نسبة الأخطاء .

- استقرار قاعدة البيانات.

الاختبار والنتائج:

بعد تشغيل اختبار الضغط بواسطة ملف:

stress-test.js

تم تشغيل ١٠٠ مستخدم في نفس الوقت.

لوحظ أن:

- النظام استمر بالاستجابة ولم يحدث انهيار للخادم .
 - عمليات الشراء المتزامنة تمت مع المحافظة على سلامة المخزون بسبب استخدام Locking.
 - لم تظهر بيانات متضاربة أو طلبات وهمية .
 - بقي زمن الاستجابة ضمن الحدود المقبولة .
- وهذا يثبت أن التصميم المستخدم قادر على التعامل مع الضغط العالي وإدارة الموارد بشكل فعال.

10_ الطلب العاشر: تحليل الاختناقات والقياس (Bottleneck Analysis & Benchmarking)

بعد تنفيذ جميع التحسينات الخاصة بالتزامن والأداء، تم إجراء تحليل لأهم نقاط الاختناق في النظام ومقارنة الأداء قبل وبعد تطبيق الحلول.

تحليل الاختناقات:

خلال التحليل تم تحديد عدة نقاط كانت تسبب بطء في النظام:

1_ كثرة الاستعلامات على قاعدة البيانات:

قبل تطبيق التخزين المؤقت، كانت صفحة عرض المنتجات تقوم بإرسال طلب مباشر إلى MySQL مع كل زيارة.

هذا يؤدي إلى:

- زيادة عدد الاستعلامات .
- ارتفاع الحمل على قاعدة البيانات .
- زيادة زمن الاستجابة.

الحل المستخدم:

Redis Cache

حيث يتم تخزين المنتجات المطلوبة وإعادة استخدامها مباشرة من الذاكرة.

2_ تنفيذ العمليات الثقيلة داخل الطلب الرئيسي:

قبل استخدام Queue كانت بعض العمليات تحتاج وقتاً طويلاً أثناء انتظار المستخدم.

الحل المستخدم:

Laravel Queue

حيث تم نقل العمليات الثقيلة إلى Worker يعمل بالخلفية، وأصبح المستخدم يحصل على استجابة مباشرة.

3_ تضارب عمليات الشراء:

قبل استخدام Locking كانت الطلبات المتزامنة قد تسبب Race Condition وتعديل غير صحيح للمخزون.

الحل المستخدم:

- Database Lock
- Redis Distributed Lock

المقارنة قبل وبعد التحسين:

العملية	قبل التحسين	بعد التحسين
عرض المنتجات	قراءة مباشرة من MySQL في كل طلب	قراءة من Redis Cache
العمليات الثقيلة	تنفيذ داخل Request وانتظار المستخدم	تنفيذ بالخلفية عبر Queue
عمليات الشراء المتزامنة	احتمال حدوث Race Condition	حماية باستخدام Locks
ضغط المستخدمين	حمل مباشر على قاعدة البيانات	توزيع ومعالجة منظمة

النتائج النهائية:

بعد تطبيق تقنيات:

- Redis Caching
- Queue Processing
- Transaction Management
- Distributed Locking
- Load Distribution

أصبح النظام:

- أسرع في عمليات القراءة .
 - أكثر استقرارًا تحت الضغط .
 - يحافظ على صحة البيانات .
 - قادرًا على التعامل مع عدد أكبر من المستخدمين المتزامنين .
- وبذلك تم تحقيق أهداف المشروع الأساسية في بناء محرك تجارة إلكترونية عالي الأداء يعتمد على مفاهيم البرمجة التفرعية وإدارة الموارد بكفاءة.